25.6.3 bridge doc discussion

Wuyunsiqin Bingsheng Zhang
Zhejiang University, CHN
3210101763@zju.edu.cn bingsheng@zju.edu.cn

May 20 2025

1 Bridge Description

1.1 Mithril Bridge

$\mathbf{Setup}_{i,j}()$

- return $\mathsf{Mithril.PGen}()$

$\mathbf{KGen}_{i,j}(pp_{i,j})$

- return $\mathsf{Mithril.KGen}(pp_{i,j})$

$\mathbf{RoT}_{i,j}(pp_{i,j}, \mathsf{vks}_{i,j}, C_i, C_j)$

- $[\mathsf{vk}_P : P \in P_{i,j}] \leftarrow \mathsf{parse}(\mathsf{keys}_{i,j})$
- $\mathsf{avk} \leftarrow \mathsf{Mithril.Avk}(pp_{i,j}, [\mathsf{vk}_P : P \in P_{i,j}])$
- $\mathsf{nonce} \leftarrow \mathsf{Beacon}(C_i)$
- put nonce in bst
- get SD from C_i
- $\mathsf{rot}_{i,j} \leftarrow (\mathsf{avk}, \mathsf{SD})$
- $\mathsf{aux}_{i,j} \leftarrow (\mathsf{rot}_{i,j}, [\mathsf{vk}_P : P \in P_{i,j}])$
- return $(\mathsf{rot}_{i,j}, \mathsf{aux}_{i,j})$

$\mathbf{Br}_{i,j}(r, m; C_i, C_j; \mathsf{bst})$ **when** $r = 0$

- get $\mathsf{rot}_{i,j}$ from bst
- $\mathsf{code} \leftarrow \mathsf{MITHBridge}[\mathsf{rot}_{i,j}].\mathsf{code}$
- $cID \leftarrow \mathsf{Hash}(\mathsf{code})$
- $\mathsf{btx}_j \leftarrow (cID, (\mathsf{code}, \mathsf{mem} := \varepsilon))$
- return $(\perp, \mathsf{btx}_j, \perp; \mathsf{bst})$

$\mathbf{Br}_{i,j}(r, m; C_i, C_j; \mathsf{bst})$ **when** $r \in [1, e-1]$

- return $(\perp, \perp, \perp; \mathsf{bst})$

$\mathbf{Br}_{i,j}(r, m; C_i, C_j; \mathsf{bst})$ **when** $r = ke - \Delta - 1$**,** $k \in N$

- get (P, sk) from bst
- get $pp_{i,j}$ from bst
- $(\mathsf{sk}', \mathsf{vk}') \leftarrow \mathsf{Mithril.KGen}(pp_{i,j})$
- put sk in bst
- $\sigma' \leftarrow \mathsf{DS.Sig}(\mathsf{sk}, \mathsf{vk}')$
- return $(\perp, \perp, (P, \mathsf{vk}', \sigma'); \mathsf{bst})$

$\mathbf{Br}_{i,j}(r, m; C_i, C_j; \mathsf{bst})$ **when** $r \in [ke - \Delta, ke - 1]$**,** $k \in N$

- put m in bst
- return $(\perp, \perp, \perp; \mathsf{bst})$

$\mathbf{Br}_{i,j}(r, m; C_i, C_j; \mathsf{bst})$ **when** $r = ke$**,** $k \in N$

- get (P, sk) from bst
- get $[\mathsf{vk}'_P : P \in P_{i,j}]$ from bst
- $\mathsf{VK}' \leftarrow [\mathsf{vk}'_P : P \in P_{i,j} \wedge \mathsf{DS.Ver}(\mathsf{vk}_P, \mathsf{vk}'_P) = 1]$
- $\mathsf{avk}' \leftarrow \mathsf{Mithril.Avk}(pp_{i,j}, \mathsf{VK}')$
- get SD' from C_i at end of round $r = ke$
- $\mathsf{cpl} \leftarrow \lfloor r/\Delta_i \rfloor$
- $h \leftarrow \mathsf{Hash}(L_i(C_i \text{ up to } \mathsf{cpl}))$
- $\mathsf{nonce}' \leftarrow \mathsf{Beacon}(C_i \text{ round } r)$
- get nonce from bst
- $\mathsf{msg.topic} \leftarrow r$
- $\mathsf{msg.body} \leftarrow (\mathsf{cpl}, h, \mathsf{SD}', \mathsf{avk}', \mathsf{nonce}')$
- put msg in bst
- get SD from C_i for round $(k-1)e$
- compute w using SD
- get w_i from SD
- $m \leftarrow [\ ]$
- for $j^* \in [m]$:
  - $\sigma \leftarrow \mathsf{Mithril.Sig}(\mathsf{sk}, j^*, \mathsf{msg}, w_i, w)$
  - if $\sigma \neq \perp$: $m \leftarrow m : (\mathsf{vk}_P, j^*, \sigma)$
- if $m = [\ ]$: $m \leftarrow \perp$
- return $(\perp, \perp, m; \mathsf{bst})$

$\mathbf{Br}_{i,j}(r, m; C_i, C_j; \mathsf{bst})$ **when** $r \in [ke+1, ke+\Delta-1]$**,** $k \in N$

- put m in bst
- return $(\perp, \perp, \perp; \mathsf{bst})$

$\mathbf{Br}_{i,j}(r, m; C_i, C_j; \mathsf{bst})$ **when** $r = ke + \Delta$**,** $k \in N$

- put m in bst
- get msg from bst
- get $[\mathsf{vk}_P : P \in P_{i,j}]$ from bst
- get SD from C_i for round $r = ke$
- get VK from bst
- form $\mathbf{WVK}$ from SD and VK
- compute w using SD
- get w_i from SD
- get $T := [(\mathsf{vk}, j^*, \sigma)]$ from bst
- $S \leftarrow [(\mathsf{vk}, j^*, \sigma) \in T : \mathsf{Mithril.Ver}(\mathsf{vk}, j^*, \mathsf{msg}, \sigma, w_i, w)]$
- $\sigma^* \leftarrow \mathsf{Mithril.ASig}(\mathbf{WVK}, \mathsf{msg}, S)$
- return $(\perp, (cID, (\mathsf{msg}, \sigma^*)), \perp; \mathsf{bst})$

$\mathbf{Br}_{i,j}(r, m; C_i, C_j; \mathsf{bst})$ **when** $r \in [ke + \Delta + 1, (k+1)e - 1]$**,** $k \in N$

- return $(\perp, \perp, \perp; \mathsf{bst})$

$\mathbf{GetVerData}_{i,j}(\mathsf{genesis}_j, \mathsf{rot}_{i,j}, L_j)$

- $\mathsf{LS} \leftarrow \mathsf{ledgerState}_j[\mathsf{genesis}_j](L_j)$
- $cID \leftarrow \mathsf{Hash}(\mathsf{MITHBridge}[\mathsf{rot}_{i,j}].\mathsf{code})$
- $(_, \mathsf{mem}, _) \leftarrow \mathsf{LS}[cID]$
- return mem

$\mathcal{R}^*_{i,j}((\mathsf{cpl}, x, h), C)$

- return $(\mathsf{Header}_i(C, \mathsf{cpl}) = h \wedge \mathcal{R}_{i,j}(x, L_i(C) \text{ up to } \mathsf{cpl}))$

$\mathbf{Certify}_{i,j}(\mathsf{rot}_{i,j}, x, C_i)$

- get $\mathsf{crs}_{i,j}$ from $\mathsf{rot}_{i,j}$
- $\mathsf{cpl} \leftarrow \lfloor r/\Delta_i \rfloor$
- $h \leftarrow \mathsf{Header}_i(C, \mathsf{cpl})$
- $\pi \leftarrow \mathsf{Snark.Prove}(\mathsf{crs}_{i,j}, (\mathsf{cpl}, x, h), C)$
- return (cpl, π)

$\mathbf{Verify}_{i,j}(\mathsf{mem}, x, \pi)$

- get $\mathsf{crs}_{i,j}$ from mem
- get list of h from mem for the cpl
- return $\mathsf{Snark.Verify}(\mathsf{crs}_{i,j}, (\mathsf{cpl}, x, h), \pi)$

Contract MITHBridge$[\mathsf{rot}_{i,j}]$ **update**(msg, σ):

- get latest avk from mem
- if $\mathsf{avk} = \perp$: get avk from $\mathsf{rot}_{i,j}$
- $(\mathsf{cpl}, h, \mathsf{SD}, \mathsf{avk}, \mathsf{nonce}) \leftarrow \mathsf{msg.body}$
- get w from SD
- check $\mathsf{Mithril.AVer}(\mathsf{avk}, \mathsf{msg}, \sigma, w)$
- $\mathsf{mem} \leftarrow \mathsf{mem} : (\mathsf{cpl}, h, \mathsf{SD}, \mathsf{avk}, \mathsf{nonce})$

verify(x, π):

- if $\mathsf{Verify}_{i,j}(\mathsf{mem}, x, \pi)$: put (ok, x, π) in mem

1.2 Mithril-Based zkBridge (SNARK Version)

Setup$_{i,j}()$

- $pp \leftarrow \mathsf{Mithril.PGen}()$
- $\mathsf{crs} \leftarrow \mathsf{Snark.Setup}()$
- return (crs, pp)

KGen$_{i,j}(pp_{i,j})$

- return $\mathsf{Mithril.KGen}(pp_{i,j})$

RoT$_{i,j}(pp_{i,j}, \mathsf{vks}_{i,j}, C_i, C_j)$

- $[\mathsf{vk}_P : P \in P_{i,j}] \leftarrow \mathsf{parse}(\mathsf{keys}_{i,j})$
- $\mathsf{avk} \leftarrow \mathsf{Mithril.Avk}(pp_i, j, [\mathsf{vk}_P : P \in \mathbf{P}_{i,j}])$
- $\mathsf{nonce} \leftarrow \mathsf{Beacon}(C_i)$
- get SD from C_i
- $\mathsf{rot}_{i,j} \leftarrow (\mathsf{avk}, \mathsf{SD}, \mathsf{crs})$
- $\mathsf{aux}_{i,j} \leftarrow (\mathsf{rot}_{i,j}, [\mathsf{vk}_P : P \in \mathbf{P}_{i,j}])$
- return $(\mathsf{rot}_{i,j}, \mathsf{aux}_{i,j})$

$\mathbf{Br}_{i,j}(r, m; C_i, C_j; \mathsf{bst})$ **when** $r = 0$

- get $\mathsf{rot}_{i,j}$ from bst
- $\mathsf{code} \leftarrow \mathsf{MITHBridge}[\mathsf{rot}_{i,j}].\mathsf{code}$
- $cID \leftarrow \mathsf{Hash}(\mathsf{code})$
- $\mathsf{btx}_j \leftarrow (cID, (\mathsf{code}, \mathsf{mem} := \varepsilon))$
- return $(\perp, \mathsf{btx}_j, \perp; \mathsf{bst})$

$\mathbf{Br}_{i,j}(r, m; C_i, C_j; \mathsf{bst})$ **when** $r \in [1, e-1]$

- return $(\perp, \perp, \perp; \mathsf{bst})$

$\mathbf{Br}_{i,j}(r, m; C_i, C_j; \mathsf{bst})$ **when** $r = ke - \Delta - 1$**,** $k \in N$

- get (P, sk) from bst
- get $pp_{i,j}$ from bst
- $(\mathsf{sk}', \mathsf{vk}') \leftarrow \mathsf{Mithril.KGen}(pp_{i,j})$
- put sk in bst
- $\sigma' \leftarrow \mathsf{DS.Sig}(\mathsf{sk}, \mathsf{vk}')$
- return $(\perp, \perp, (P, \mathsf{vk}', \sigma'); \mathsf{bst})$

$\mathbf{Br}_{i,j}(r, m; C_i, C_j; \mathsf{bst})$ **when** $r \in [ke - \Delta, ke - 1]$**,** $k \in N$

- put m in bst
- return $(\perp, \perp, \perp; \mathsf{bst})$

$\mathbf{Br}_{i,j}(r, m; C_i, C_j; \mathsf{bst})$ **when** $r = ke$**,** $k \in N$

- get (P, sk) from bst
- get $[\mathsf{vk}'_P : P \in P_{i,j}]$ from bst
- $\mathsf{VK}' \leftarrow [\mathsf{vk}'_P : P \in P_{i,j} \wedge \mathsf{DS.Ver}(\mathsf{vk}_P, \mathsf{vk}'_P) = 1]$
- $\mathsf{avk}' \leftarrow \mathsf{Mithril.Avk}(pp_{i,j}, \mathsf{VK}')$
- get SD' from C_i at end of round $r = ke$
- $\mathsf{cpl} \leftarrow \lfloor r/\Delta_i \rfloor$
- $h \leftarrow \mathsf{Hash}(L_i(C_i \text{ up to } \mathsf{cpl}))$
- $\mathsf{nonce}' \leftarrow \mathsf{Beacon}(C_i \text{ round } r)$
- get nonce from bst
- $\mathsf{msg.topic} \leftarrow r$
- $\mathsf{msg.body} \leftarrow (\mathsf{cpl}, h, \mathsf{SD}', \mathsf{avk}', \mathsf{nonce}')$
- put msg in bst
- get SD from C_i for round $(k-1)e$
- compute w using SD
- get w_i from SD
- $m \leftarrow [\ ]$
- for $j^* \in [m]$:
  - $\sigma \leftarrow \mathsf{Mithril.Sig}(\mathsf{sk}, j^*, \mathsf{msg}, w_i, w)$
  - if $\sigma \neq \perp$: $m \leftarrow m : (\mathsf{vk}_P, j^*, \sigma)$
- if $m = [\ ]$: $m \leftarrow \perp$
- return $(\perp, \perp, m; \mathsf{bst})$

$\mathbf{Br}_{i,j}(r, m; C_i, C_j; \mathsf{bst})$ **when** $r \in [ke+1, ke+\Delta-1]$**,** $k \in N$

- put m in bst
- return $(\perp, \perp, \perp; \mathsf{bst})$

$\mathbf{Br}_{i,j}(r, m; C_i, C_j; \mathsf{bst})$ **when** $r = ke + \Delta$**,** $k \in N$

- put m in bst
- get msg from bst
- get $[\mathsf{vk}_P : P \in P_{i,j}]$ from bst
- get SD from C_i for round $r = ke$
- get VK from bst
- get $\mathbf{WVK}$ from SD and VK
- compute w using SD
- get w_i from SD
- get $T := [(\mathsf{vk}, j^*, \sigma)]$ from bst
- $\mathbf{S} \leftarrow [(\mathsf{vk}, j^*, \sigma) \in T : \mathsf{Mithril.Ver}(\mathsf{vk}, j^*, \mathsf{msg}, \sigma, w_i, w)]$
- $\sigma^* \leftarrow \mathsf{Mithril.ASig}(\mathbf{WVK}, \mathsf{msg}, \mathbf{S})$
- get crs from bst
- $\pi^* \leftarrow \mathsf{Snark.Prove}(\mathsf{crs}, (\mathsf{msg}, \sigma^*), (\mathbf{WVK}, \mathbf{S}))$
- return $(\perp, (cID, ((\mathsf{msg}, \sigma*), \pi^*)), \perp; \mathsf{bst})$

$\mathbf{Br}_{i,j}(r, m; C_i, C_j; \mathsf{bst})$ **when** $r \in [ke + \Delta + 1, (k+1)e - 1]$**,** $k \in N$

- return $(\perp, \perp, \perp; \mathsf{bst})$

$\mathbf{GetVerData}_{i,j}(\mathsf{genesis}_j, \mathsf{rot}_{i,j}, L_j)$

- $\mathsf{LS} \leftarrow \mathsf{ledgerState}_j[\mathsf{genesis}_j](L_j)$
- $cID \leftarrow \mathsf{Hash}(\mathsf{MITHBridge}[\mathsf{rot}_{i,j}].\mathsf{code})$
- $(_, \mathsf{mem}, _) \leftarrow \mathsf{LS}[cID]$
- return mem

$\mathcal{R}^*_{i,j}((\mathsf{cpl}, x, h), C)$

- return $(\mathsf{Header}_i(C, \mathsf{cpl}) = h \wedge \mathcal{R}_{i,j}(x, L_i(C) \text{ up to } \mathsf{cpl}))$

$\mathbf{Certify}_{i,j}(\mathsf{rot}_{i,j}, x, C_i)$

- get $\mathsf{crs}_{i,j}$ from $\mathsf{rot}_{i,j}$
- $\mathsf{cpl} \leftarrow \lfloor r/\Delta_i \rfloor$
- $h \leftarrow \mathsf{Header}_i(C, \mathsf{cpl})$
- $\pi \leftarrow \mathsf{Snark.Prove}(\mathsf{crs}_{i,j}, (\mathsf{cpl}, x, h), C)$
- return (cpl, π)

$\mathbf{Verify}_{i,j}(\mathsf{mem}, x, \pi)$

- get $\mathsf{crs}_{i,j}$ from mem
- get list of h from mem for the cpl
- return $\mathsf{Snark.Verify}(\mathsf{crs}_{i,j}, (\mathsf{cpl}, x, h), \pi)$

Contract: MITHBridge$[\mathsf{rot}_i, j]$ **update**(msg, π):

1. get crs from $\mathsf{rot}_{i,j}$
2. check $\mathsf{Snark.Verify}(\mathsf{crs}, (\mathsf{msg}, \sigma*), \pi^*)$
3. parse $(\mathsf{cpl}, h, \mathsf{SD}, \mathsf{avk}, \mathsf{nonce}) \leftarrow \mathsf{msg.body}$
4. $\mathsf{mem} \leftarrow \mathsf{mem} : (\mathsf{cpl}, h, \mathsf{SD}, \mathsf{avk}, \mathsf{nonce})$

verify(x, π):

1. if $\mathsf{Verify}_{i,j}(\mathsf{mem}, x, \pi)$: put (ok, x, π) in mem

The following are the core algorithms that require zero-knowledge proof in the Mithril-based zero-knowledge proof cross-chain bridge:

Mithril Protocol Π_{Mithril}

Inputs: Merkle tree commitment of the committee public keys $\langle AVK \rangle$, aggregate public key $(ivk, ivk_{\mathsf{body}})$, aggregate signature (μ, e_σ), verification information $\{(mvk_i, stake_i, p_i, ev_i, \sigma_i, index_i)\}_{i \in S}$, and message m.

$\mathsf{Verify}(\langle AVK \rangle, ivk, ivk_{\mathsf{body}}, \mu, e_\sigma, \{(mvk_i, stake_i, p_i, ev_i, \sigma_i, index_i)\}_{i \in S}, m, k) \to 0/1$:

- $ivk = \mathsf{MSP.BKey}(\mathbf{mvk}, e_\sigma)$, and $ivk_{\mathsf{body}} = \mathsf{MSP.AKey}(\mathbf{mvk})$ ▷ Aggregate key derivation
- $(\mu, e_\sigma) = \mathsf{MSP.BSig}(\boldsymbol{\sigma})$ ▷ Aggregate signature and weight seed derivation
- For all i: $index_i \leq m$ and for all $i \neq j$: $index_i \neq index_j$ ▷ Index range and uniqueness checks
- For all $i \in \{1, \ldots, k\}$: verify that $(mvk_i, stake_i)$ is included in Merkle tree $\langle AVK \rangle$ along path p_i ▷ Merkle membership check
- For all $i \in \{1, \ldots, k\}$: verify $\mathsf{MSP.Eval}(topic, index_i, \sigma_i) = ev_i$ ▷ Eligibility value computation
- For all $i \in \{1, \ldots, k\}$: verify $ev_i \leq \phi(stake_i)$ ▷ Weight threshold check
- If any of the above checks fail, output 0; otherwise, output 1

2 Benchmark Result on the New Server

2.1 Server Configuration Information

The experiments were conducted on a G5500 V6 Whitley high-performance computing server, equipped with an Intel Xeon Platinum 8368 processor (152 cores @ 3.4GHz), 256GB of RAM, and running Ubuntu 22.04.5 LTS with kernel version 5.15.0-131-generic.

2.2 Benchmark Result

Verification Relations for MSP.A:

- $\mathsf{MSP.AKey}(\mathbf{mvk})$ receives a set of verification keys $\mathbf{mvk}$ and computes the aggregate key $ivk = \sum_1^d mvk_i$.
- $\mathsf{MSP.ASig}(\sigma)$ aggregates signatures as $\mu = \sum_1^d \sigma_i$.
- $\mathsf{MSP.Ver}(msg, ivk, \mu)$ checks if $e(\mu, g_2) = e(H_{G_1}(\text{“M”}\|msg), ivk)$.

Table 1: Benchmark Results of MSP.A

Aggregated sig num	**2**	**200**	**2000**	**4000**	**6000**	**8000**
Proof Time (s)	19.17	29.51	72.89	123.67	181.38	226.84
Proof Size (bytes)	3072	4128	11424	18944	26944	34944
Verification Time (ms)	20.93	9.09	19.95	16.03	20.25	55.16

Verification Relations for MSP.B:

- $\mathsf{MSP.BKey}(\mathbf{mvk}, \mathbf{e}_\sigma)$ computes $ivk = \sum mvk_i^{e_i}$, where $e_i \leftarrow H_\lambda(i, \mathbf{e}_\sigma)$.
- $\mathsf{MSP.BSig}(\sigma)$ returns $(\mu, \mathbf{e}_\sigma)$, with $\mu = \sum \sigma_i^{e_i}$.
- $\mathsf{MSP.Ver}(msg, ivk, \mu)$ verifies $e(\mu, g_2) = e(H_{G_1}(\text{“M”}\|msg), ivk)$.

Table 2: Benchmark Results of MSP.B

Aggregated sig num	**8**	**16**	**32**	**64**	**128**	**256**
Proof Time (s)	35.73	48.48	65.37	103.75	184.02	327.30
Proof Size (bytes)	5856	7936	10944	17312	30240	55744
Verification Time (ms)	10.30	14.68	15.05	14.54	24.51	69.35

Validation Relation: For all i, $\mathsf{index}_i \leq m$ and for all $i \neq j$, $\mathsf{index}_i \neq \mathsf{index}_j$.

Table 3: Benchmark Results of Index Validation

Number of Index Checks	**128**	**256**	**512**	**1024**	**2048**
Proof Time (s)	4.08	6.77	13.41	37.39	122.49
Proof Size (bytes)	960	1920	4576	14944	14944
Verification Time (ms)	5.05	6.15	9.42	15.10	13.08

Verification Relation: For each $i \in \{1, \ldots, k\}$,

$$\mathsf{MSP.Eval}(\mathsf{topic}, \mathsf{index}_i, \sigma_i) = ev_i \quad \text{and} \quad ev_i \leq \phi(\mathsf{stake}_i).$$

Table 4: Benchmark Results of Eligibility Check

Number of Checks	**128**	**256**	**512**	**1024**	**2048**
Proof Time (s)	10.14	12.78	17.36	26.39	44.56
Proof Size (bytes)	1344	1920	2848	4800	8832
Verification Time (ms)	6.77	6.39	7.10	11.24	12.59